

SIEM Security Dashboard

15-Day Hands-On Project Report

Project Details	Information
Student Name	Jatin Mehta
Batch / Cohort	Cybersecurity Training Batch
Domain	Network Security & Event Monitoring
Trainer Name	Ganesh Kuche
Submission Date	08 / 10 / 2026
Difficulty Level	Intermediate

SECTION 1 — PROJECT ABSTRACT

1. Project Abstract

SIEM Security Dashboard is a cybersecurity monitoring and security event management application designed to collect, process, analyze, and visualize security logs from a centralized system. The project addresses the challenge of unorganized raw logs by converting them into structured, normalized data stored in a SQLite database. Using a rule-based detection engine, the system identifies suspicious activities such as repeated failed login attempts (brute-force attacks) and unauthorized file access. It automatically generates severity-classified alerts (HIGH, MEDIUM) and tracks source IPs in real time. Built using FastAPI for the backend, SQLAlchemy ORM, and React with Vite for the interactive frontend dashboard, it provides a lightweight SOC-style monitoring interface along with Swagger/OpenAPI interactive testing documentation.

SECTION 2 — INTRODUCTION

2. Introduction

2.1 Background

In today's digital landscape, enterprise systems and network devices generate millions of raw security logs daily. Analyzing these logs manually is slow, inefficient, and highly prone to missing critical security incidents. Security Information and Event Management (SIEM) systems solve this issue by providing a centralized platform to collect, aggregate, and analyze security events across an enterprise infrastructure. Modern Security Operations Centers (SOCs) heavily rely on SIEM tools to detect active threats, ensure regulatory compliance, and respond rapidly to potential cyberattacks.

2.2 Problem Statement

Traditional log monitoring becomes extremely difficult when security events are stored in raw, unstructured, and fragmented formats across different servers. Security analysts often struggle to efficiently examine authentication attempts, source IP addresses, event types, and suspicious activities due to high log volume. Without a centralized rule-based system, brute-force login attempts and unauthorized file access can go unnoticed until significant damage occurs.

2.3 Objectives

- To collect, ingest, parse, and normalize raw security log data into structured formats.
- To store processed security logs in a centralized SQLite database using SQLAlchemy ORM.
- To implement a detection engine that identifies repeated failed login attempts (brute-force) and suspicious file access.
- To generate security alerts classified by severity levels (HIGH, MEDIUM) and track source IP activity.
- To provide search, event filtering, REST API endpoints, and an interactive React-based dashboard for SOC-style monitoring.

SECTION 3 — TOOLS & ENVIRONMENT

3. Tools and Environment Setup

Tool / Software	Version / Platform	Purpose
Python	3.10+	Primary backend programming language
FastAPI	Latest	Web framework for high-performance REST APIs
SQLite & SQLAlchemy	SQLite 3 / SQLAlchemy ORM	Centralized database storage and Object Relational Mapping
React & Vite	React 18+ / Vite	Frontend framework for interactive dashboard visualization
Axios	Latest	HTTP client for backend API communications
Swagger / OpenAPI	OAS 3.1	Interactive REST API documentation and endpoint testing
VS Code & Git	VS Code / GitHub	Development environment and version control

3.1 Lab / Environment Description

The lab environment was set up locally on a development host machine using Visual Studio Code as the primary IDE. The backend environment runs on Python using FastAPI with SQLite database integration via SQLAlchemy. The frontend dashboard was built using React powered by Vite for rapid UI rendering and Axios for real-time asynchronous API requests. API endpoints were tested directly using FastAPI's built-in Swagger UI framework ([/docs](#)).

SECTION 4 — METHODOLOGY / STEP-BY-STEP PROCESS

4. Methodology

Step 1 — Log Ingestion & Data Parsing

Raw log files containing system authentication and file access records are ingested into the backend system. The parser reads the unformatted text logs and splits them into structured data attributes: Timestamp, Event Type, Username, Source IP, Status, and Message

Endpoint: POST /logs/ingest

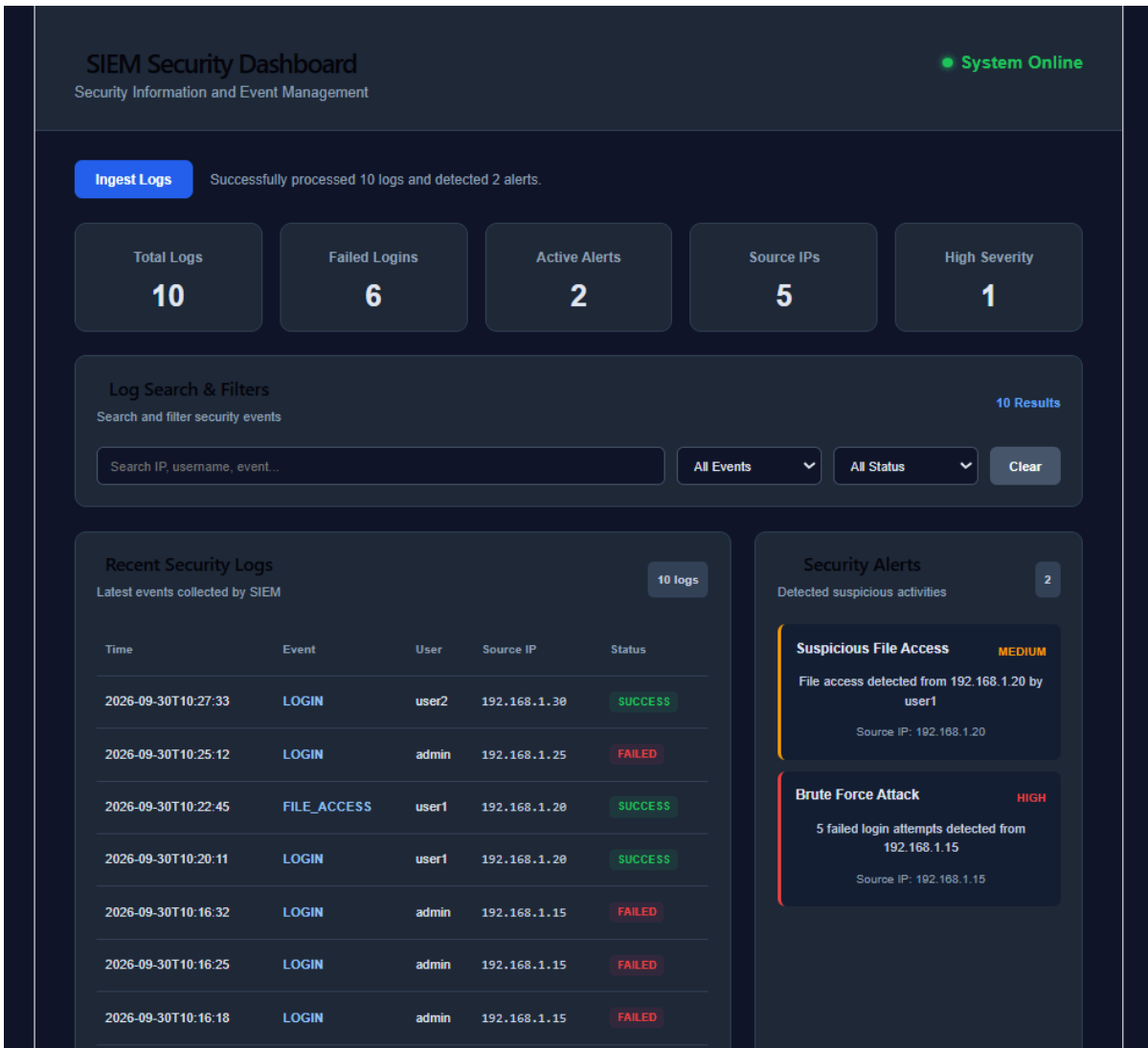


Figure 1: SIEM Security Dashboard showing overall stats after log ingestion.

Step 2 — Log Normalization & Database Storage

Parsed log fields are standardized into uniform data formats so that the detection rules can process them accurately. The normalized records are stored persistently into the SQLite database using SQLAlchemy ORM.

Step 3 — Rule-Based Detection Engine Execution

The detection engine continuously evaluates stored log entries against predefined security rules:

- Brute-Force Rule: 5 or more failed login attempts from the same Source IP → Triggers HIGH Severity Alert.
- File Access Rule: Specific user file-access patterns on restricted files → Triggers MEDIUM Severity Alert.

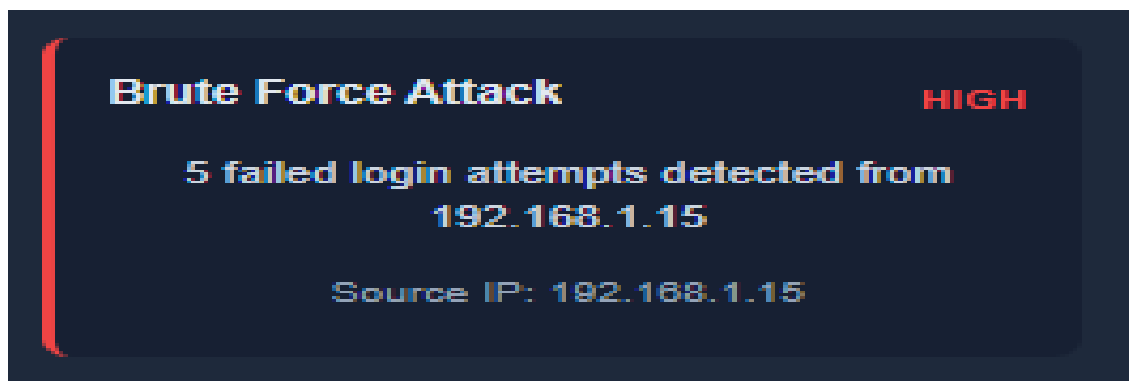


Figure 2: Generated HIGH Severity Brute Force Attack alert for IP 192.168.1.15 due to 5 failed login attempts.

Step 4 — Log Search & Investigation Filtering

Analysts can investigate security events on the React dashboard using IP addresses, usernames, or event types. Filtering isolates specific events like FILE_ACCESS or specific attacker IPs like 192.168.1.15.

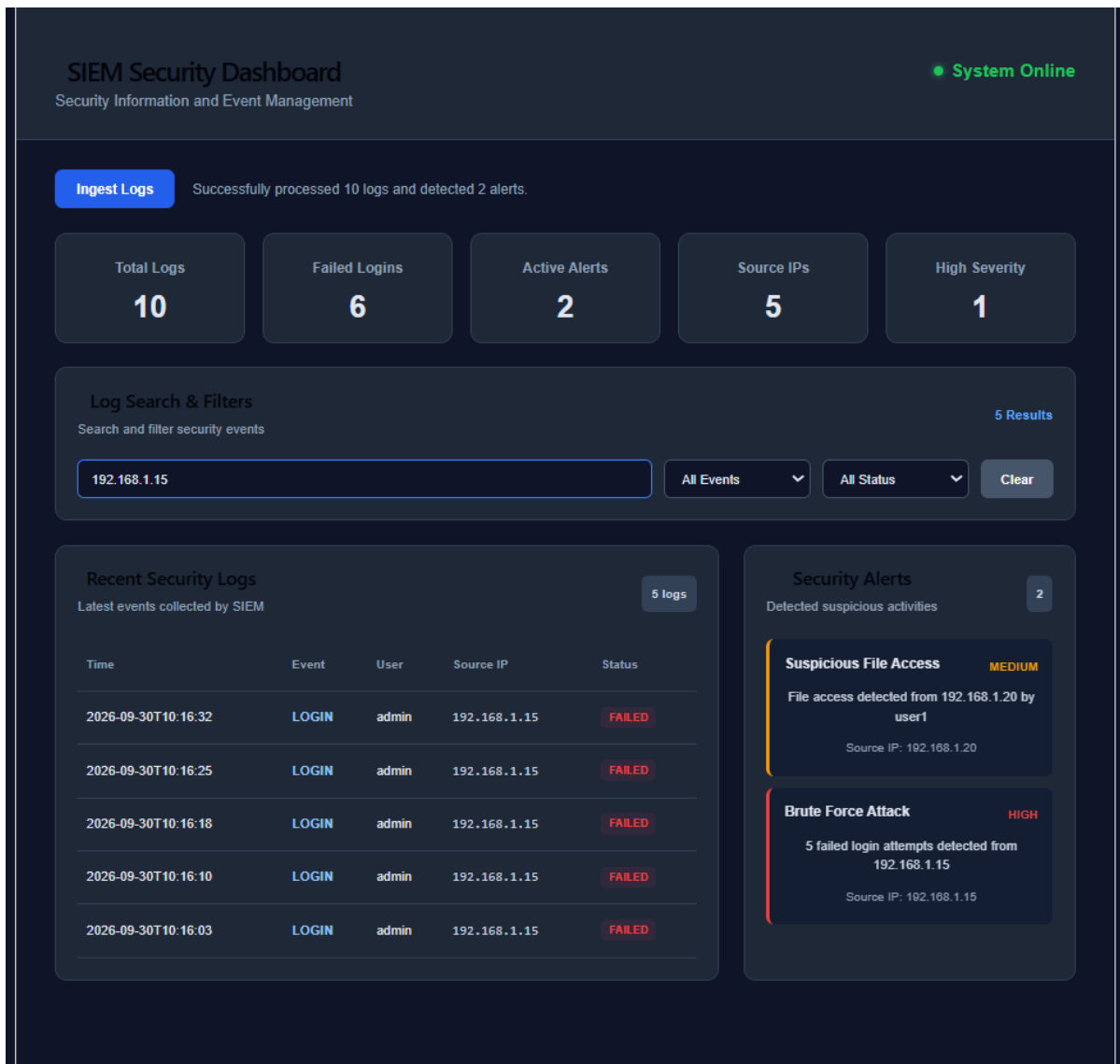


Figure 3: Filtering logs by Source IP (192.168.1.15) to isolate and analyze failed authentication attempts.

Step 5 — REST API & Swagger UI Testing

All backend capabilities (log ingestion, log fetching, alert queries, health checks) are exposed via RESTful endpoints and verified using Swagger UI.

SIEM Security Dashboard ^{1.0.0} ^{OAS 3.1}

/openapi.json

Cybersecurity monitoring and security event management system

Logs ^	
POST	/logs/ingest Ingest Logs ▾
GET	/logs/ Get Logs ▾
GET	/logs/alerts Get Alerts ▾
default ^	
GET	/ Root ▾
GET	/health Health Check ▾

Figure 4: Interactive FastAPI Swagger UI documentation showing endpoints like /logs/ingest, /logs/, and /logs/alerts.

SECTION 5 — RESULTS & OBSERVATIONS

5. Results and Observations

5.1 Summary of Findings

- **Brute Force Detection** | 5 consecutive failed login attempts detected from IP 192.168.1.15 | Severity: HIGH
- **Suspicious File Access** | Unauthorized file access detected from IP 192.168.1.20 by user1 | Severity: MEDIUM
- **Log Processing Count** | Successfully processed a batch of 10 security logs containing 6 failed login attempts | Severity: Info
- **Source IP Tracking** | System tracked 5 unique source IPs across all system log entries | Severity: Info

5.2 Analysis

During testing, the system demonstrated high efficiency in isolating security threats from raw log data. Upon processing 10 sample logs, the rule engine immediately flagged IP 192.168.1.15 for attempting multiple failed logins within a short timeframe, raising a HIGH severity alert. The investigation filter verified that user admin was targeted 5 times consecutively. Furthermore, the system captured a FILE_ACCESS event by user1 from IP

192.168.1.20, raising a MEDIUM severity alert. This proves that lightweight rule engines can effectively mitigate automated credential stuffing and insider threat activities.

SECTION 6 — CHALLENGES FACED

6. Challenges Faced

- **Challenge 1: Unstructured Log Format Parsing** → Implemented regex-based parsing scripts in Python to extract timestamp, IP, status, and username accurately.
- **Challenge 2: Duplicate Alert Generation** → Added status state checks in SQLAlchemy to ensure alerts are generated only once per detection session.
- **Challenge 3: Real-time UI Sync with API** → Integrated Axios asynchronous state handling in React to update dashboard statistics dynamically after log ingestion.

SECTION 7 — LEARNING OUTCOMES

7. Learning Outcomes

- **SIEM & SOC Principles:** Understood fundamental workflows of log collection, parsing, normalization, and centralized event monitoring.
- **Rule Engine Implementation:** Implemented threshold-based detection logic for brute-force attacks and file access anomalies.
- **Backend API Development:** Built RESTful APIs using Python, FastAPI, and SQLAlchemy ORM.
- **Frontend Visualization:** Created an interactive SOC security dashboard using React, Vite, and Axios.
- **API Documentation & Testing:** Mastered API endpoint testing and schema validation using Swagger/OpenAPI.

SECTION 8 — CONCLUSION & FUTURE SCOPE

8. Conclusion

The SIEM Security Dashboard project successfully demonstrated the practical implementation of a lightweight SIEM and SOC-style security monitoring application. The system fulfilled all primary objectives, including log parsing, normalization, database persistence, rule-based threat detection, and real-time visualization on an interactive dashboard. It effectively flagged brute-force attempts and suspicious file access, giving security analysts an easy-to-use search and filtering interface for incident response.

8.1 Future Scope

- **Real-time Streaming:** Integration of WebSockets or Kafka for live streaming log ingestion.
- **Threat Intelligence:** Cross-referencing source IPs with external threat intelligence feeds (e.g., AbuseIPDB, VirusTotal).
- **Role-Based Access Control (RBAC):** Adding authentication and multi-user authorization for SOC tier analysts.
- **Automated Notifications:** Integrating Email, Slack, or Webhook notifications for HIGH severity alerts.

SECTION 9 — REFERENCES

9. References

- [1] FastAPI Documentation — <https://fastapi.tiangolo.com/>
- [2] React & Vite Setup Guide — <https://vitejs.dev/>
- [3] SQLAlchemy Documentation — <https://www.sqlalchemy.org/>
- [4] SIEM Architecture Basics (MITRE ATT&CK) — <https://attack.mitre.org/>

DECLARATION

Student Declaration

I hereby declare that this project report is my own original work, completed as part of the Labmentix Cybersecurity Training Program. The work presented in this report is genuine, and all external references have been duly acknowledged.

Student Signature: Jatin Mehta

Date of Submission: 08 / 10 / 2026